

RLS

22.46 Procesamiento Adaptativo de Señales Aleatorias

Marc S. Ressl

Algoritmos LS recursivos

Hoy aplicaremos el método de least-squares al desarrollo de algoritmos **recursivos**.

Su principal ventaja es la disminución del costo computacional de least-squares.

Algoritmos LS recursivos

En todos estos algoritmos utilizamos la función de costo:

$$E(n) = \sum_{j=0}^{n} \lambda^{n-j} |e(j)|^2$$

La constante λ, $0 < \lambda \leq 1$, es un **factor de olvido**.

Los algoritmos LS recursivos son, por tanto, algoritmos LS pesados.

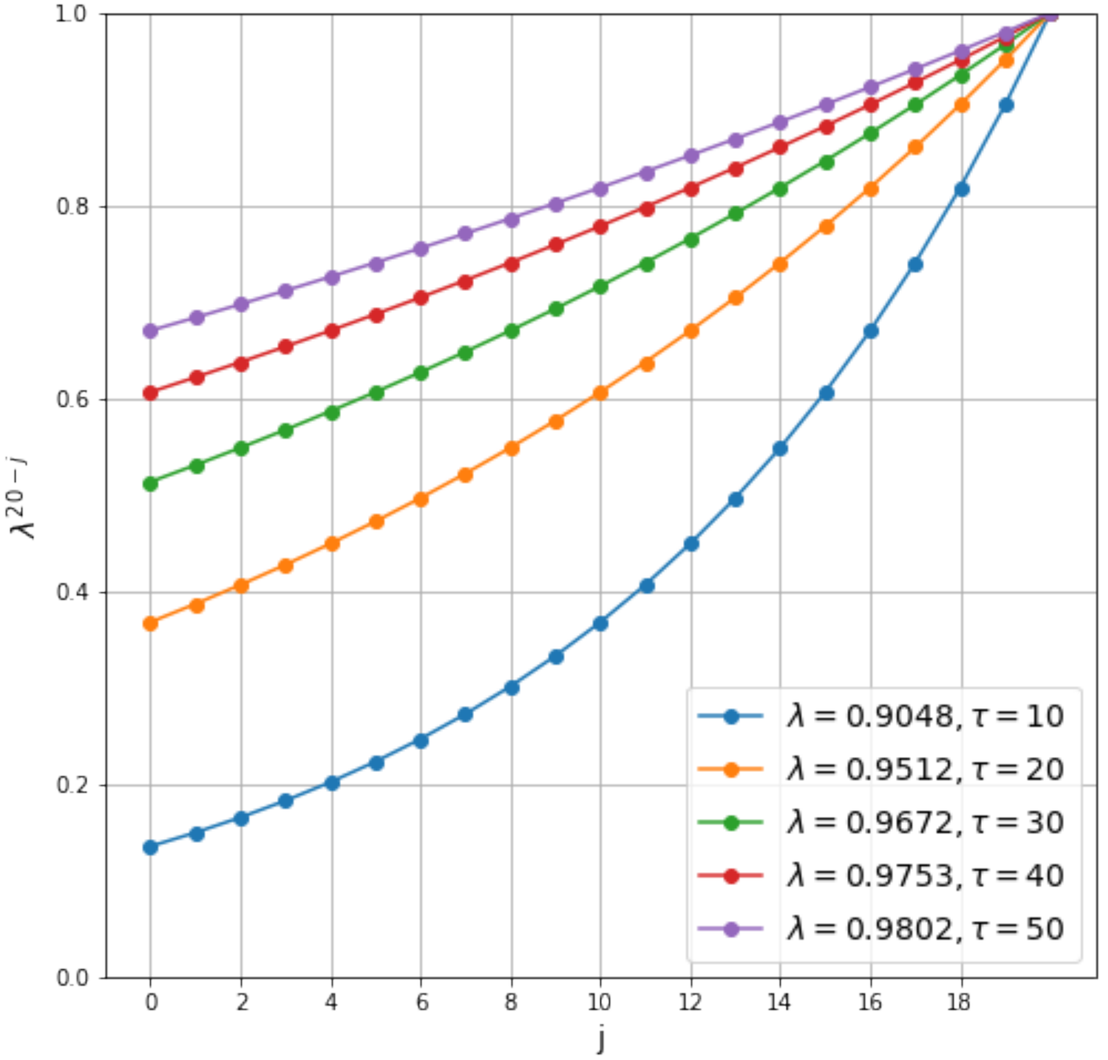
Algoritmos LS recursivos

La función de peso es la exponencial:

$$f_j(n) = \lambda^{n-j}$$

La constante de tiempo, en número de muestras, es:

$$\tau = -\frac{1}{\ln(\lambda)}$$

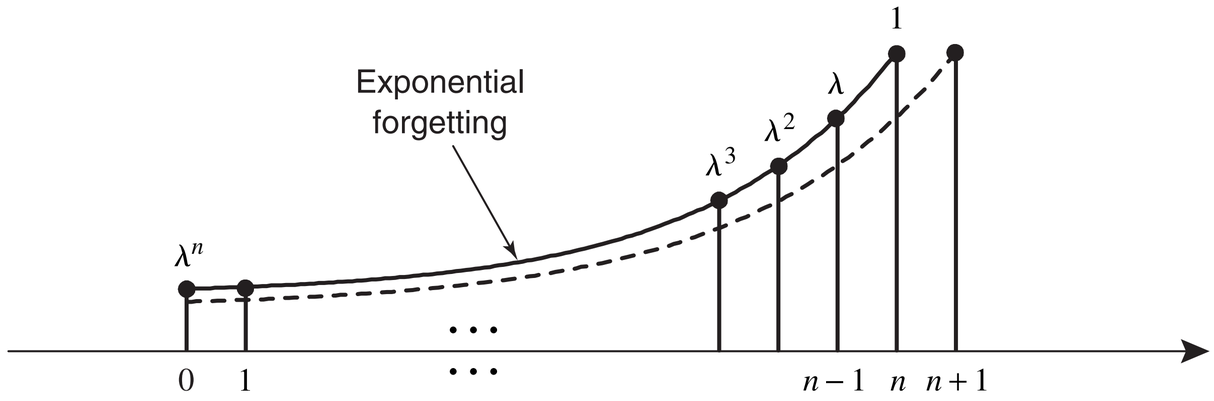


Algoritmos LS recursivos

El factor de olvido reduce la importancia de los datos del pasado remoto (permite “olvidarlos”).

Controla, por tanto, la **capacidad de tracking** del algoritmo.

Cuando $\lambda = 1$, el algoritmo tiene **memoria infinita** porque el vector de coeficientes $\mathbf{w}(n)$ depende por igual de todos los valores pasados.



Algoritmos LS recursivos

Los coeficientes $\mathbf{w}(n)$ óptimos se obtienen con:

$$\hat{\mathbf{R}}(n)\mathbf{w}(n) = \hat{\mathbf{p}}(n)$$

En donde:

$$\hat{\mathbf{R}}(n) = \sum_{j=0}^{n} \lambda^{n-j}\mathbf{x}(j)\mathbf{x}^{H}(j) \qquad \hat{\mathbf{p}}(n) = \sum_{j=0}^{n} \lambda^{n-j}\mathbf{x}(j)y^{*}(j)$$

Algoritmos LS recursivos

El error óptimo es:

$$E_o(n) = E_y(n) - \hat{\mathbf{p}}^H(n)\mathbf{w}(n)$$

$E_y(n)$ es la energía de la señal deseada pesada:

$$E_y(n) = \sum_{j=0}^{n} \lambda^{n-j}|y(j)|^2$$

Algoritmos LS recursivos

Con esta función de costo, se verifica que:

$$\hat{\mathbf{R}}(n) = \lambda \hat{\mathbf{R}}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n)$$

Dado que la matriz $\mathbf{x}(n)\mathbf{x}^H(n)$ es de rango 1, la actualización de $\mathbf{R}(n)$ es una modificación de rango 1.

Algoritmos LS recursivos

En modo análogo:

$$\hat{\mathbf{p}}(n) = \lambda \hat{\mathbf{p}}(n-1) + \mathbf{x}(n)y^*(n)$$

Algoritmos LS recursivos

Esto nos da la pauta de que existen **relaciones de recurrencia** que nos permiten calcular el vector de coeficientes $\mathbf{w}(n)$ a partir de $\mathbf{w}(n - 1)$ y las nuevas observaciones $\{\mathbf{x}(n), y(n)\}$.

Algoritmos LS recursivos

Existen dos formas para realizar esta actualización:

- Con el algoritmo LS recursivo **a priori**.
- Con el algoritmo LS recursivo **a posteriori**.

Los algoritmos son equivalentes y producen el mismo resultado, difiriendo sólo en su forma.

Algoritmo LS recursivo a priori

Para desarrollar el **algoritmo LS recursivo a priori** resolvemos las relaciones de recurrencia en términos de $\hat{\mathbf{R}}(n-1)$ y $\hat{\mathbf{p}}(n-1)$, y las introducimos en las ecuaciones normales:

$$\left[\hat{\mathbf{R}}(n) - \mathbf{x}(n)\mathbf{x}^H(n)\right]\mathbf{w}(n-1) = \hat{\mathbf{p}}(n) - \mathbf{x}(n)y^*(n)$$

Algoritmo LS recursivo a priori

Podemos re-escribir esta igualdad como:

$$\hat{\mathbf{R}}(n)\mathbf{w}(n-1) + \mathbf{x}(n)e^*(n) = \hat{\mathbf{p}}(n)$$

En donde $e(n)$ es el **error de estimación a priori**:

$$e(n) = y(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$$

Algoritmo LS recursivo a priori

Si $\hat{\mathbf{R}}(n)$ es invertible, también podemos multiplicar ambos lados de la ecuación anterior por $\hat{\mathbf{R}}^{-1}(n)$.

Utilizando nuevamente las ecuaciones normales, obtenemos:

$$\mathbf{w}(n-1) + \hat{\mathbf{R}}^{-1}(n)\mathbf{x}(n)e^{*}(n) = \hat{\mathbf{R}}^{-1}(n)\hat{\mathbf{p}}(n) = \mathbf{w}(n)$$

Algoritmo LS recursivo a priori

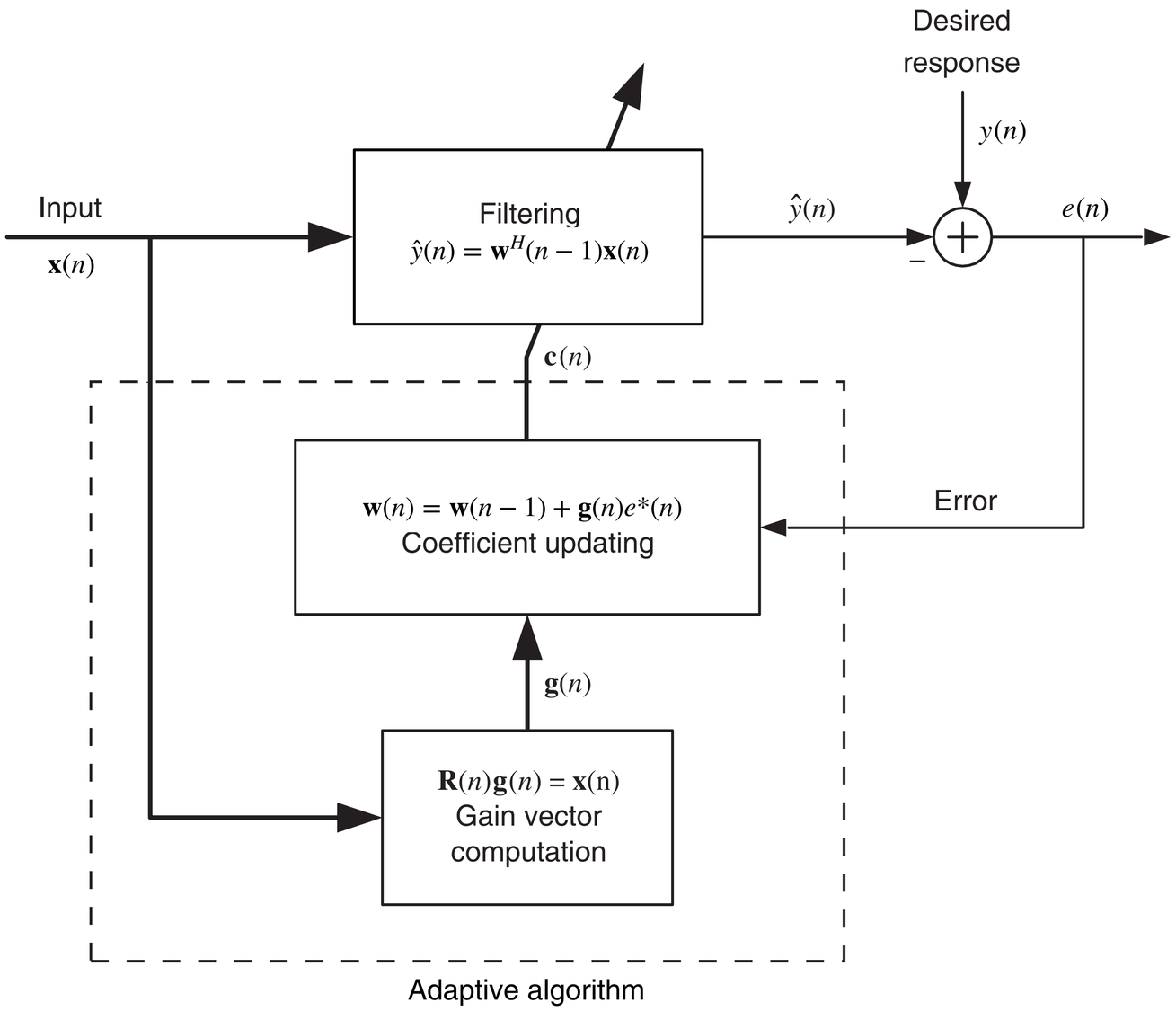
Definiendo el **vector de ganancia de adaptación** $\mathbf{g}(n)$:

$$\hat{\mathbf{R}}(n)\mathbf{g}(n) = \mathbf{x}(n)$$

Llegamos a la **ecuación de actualización** del algoritmo LS recursivo a priori:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{g}(n)e^{*}(n)$$

Esquema


Desired
response
$y(n)$
Input
$\mathbf{x}(n)$
Filtering
$\hat{y}(n) = \mathbf{w}^H(n-1)\mathbf{x}(n)$
$\hat{y}(n)$
$e(n)$
$\mathbf{c}(n)$
$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{g}(n)e^*(n)$
Coefficient updating
Error
$\mathbf{g}(n)$
$\mathbf{R}(n)\mathbf{g}(n) = \mathbf{x}(n)$
Gain vector
computation
Adaptive algorithm

Algoritmo LS recursivo a posteriori

Para desarrollar el **algoritmo LS recursivo a posteriori** introducimos las relaciones de recurrencia directamente en las ecuaciones normales. Operando:

$$\lambda\hat{\mathbf{R}}(n-1)\mathbf{w}(n) - \mathbf{x}(n)\varepsilon^*(n) = \lambda\hat{\mathbf{p}}(n-1)$$

En donde $\varepsilon(n)$ es el **error de estimación a posteriori**:

$$\varepsilon(n) = y(n) - \mathbf{w}^H(n)\mathbf{x}(n)$$

Algoritmo LS recursivo a posteriori

Si $\hat{\mathbf{R}}(n-1)$ es invertible, podemos multiplicar ambos lados de la ecuación anterior por $\lambda^{-1}\hat{\mathbf{R}}^{-1}(n-1)$.
Obtenemos:

$$\mathbf{w}(n) - \lambda^{-1}\hat{\mathbf{R}}^{-1}(n-1)\mathbf{x}(n)\varepsilon^*(n) = \hat{\mathbf{R}}^{-1}(n-1)\hat{\mathbf{p}}(n-1) = \mathbf{w}(n-1)$$

Algoritmo LS recursivo a posteriori

Definiendo el **vector de ganancia de adaptación alternativo** $\bar{\mathbf{g}}(n)$:

$$\lambda\hat{\mathbf{R}}(n-1)\bar{\mathbf{g}}(n) = \mathbf{x}(n)$$

Llegamos a la **ecuación de actualización** del algoritmo LS recursivo a posteriori:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \bar{\mathbf{g}}(n)\varepsilon^*(n)$$

Algoritmo LS recursivo a posteriori

Sin embargo no podemos realizar el algoritmo en esta forma porque la expresión de $\varepsilon(n)$ está acoplada a la ecuación de actualización.

Algoritmo LS recursivo a posteriori

Para realizar el algoritmo, introducimos la ecuación de actualización de $\mathbf{w}(n)$ en la expresión de $\varepsilon(n)$:

$$\begin{aligned}\varepsilon(n) &= y(n) - \left[\mathbf{w}^H(n-1) + \bar{\mathbf{g}}^H(n)\varepsilon^*(n)\right]\mathbf{x}(n) \\ &= e(n) - \varepsilon(n)\bar{\mathbf{g}}^H(n)\mathbf{x}(n)\end{aligned}$$

Algoritmo LS recursivo a posteriori

Y simplificamos en:

$$\varepsilon(n) = \frac{e(n)}{\bar{\alpha}(n)}$$

$\bar{\alpha}(n)$ se conoce como **factor de conversión**:

$$\begin{aligned}\bar{\alpha}(n) &= 1 + \bar{\mathbf{g}}^H(n)\mathbf{x}(n) \\ &= 1 + \lambda^{-1}\mathbf{x}^H(n)\hat{\mathbf{R}}^{-1}(n-1)\mathbf{x}(n)\end{aligned}$$

Algoritmo LS recursivo a posteriori

El factor de conversión $\bar{\alpha}(n)$ permite transformar el error a priori $e(n)$ en el error a posteriori $\varepsilon(n)$.

Este truco hace posible la actualización del vector de coeficientes, y por tanto hace posible la realización del algoritmo LS a posteriori.

Algoritmo LS recursivo a posteriori

Comparando la ecuación de actualización de LS recursivo a priori con la ecuación de actualización LS a posteriori, descubrimos que:

$$\mathbf{g}(n) = \frac{\bar{\mathbf{g}}(n)}{\bar{\alpha}(n)}$$

$\mathbf{g}(n)$ y $\bar{\mathbf{g}}(n)$ tienen, por tanto, la misma dirección, pero diferente longitud.

Algoritmo LS recursivo a posteriori

También podemos definir un factor de conversión $\alpha(n)$ para el algoritmo LS a priori:

$$\alpha(n) = 1 - \mathbf{x}^H(n)\mathbf{R}^{-1}(n)\mathbf{x}(n) = 1 - \mathbf{g}^H(n)\mathbf{x}(n)$$

Es fácil probar que:

$$\alpha(n) = \frac{1}{\bar{\alpha}(n)}$$

Resumen

	A priori LS adaptive filter	**A posteriori LS adaptive filter**
Correlation matrix	$\hat{\mathbf{R}}(n) = \lambda\hat{\mathbf{R}}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n)$	$\hat{\mathbf{R}}(n) = \lambda\hat{\mathbf{R}}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n)$
Adaptation gain	$\hat{\mathbf{R}}(n)\mathbf{g}(n) = \mathbf{x}(n)$	$\lambda\hat{\mathbf{R}}(n-1)\bar{\mathbf{g}}(n) = \mathbf{x}(n)$
A priori error	$e(n) = y(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$	$e(n) = y(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$
Conversion factor	$\alpha(n) = 1 - \mathbf{g}^H(n)\mathbf{x}(n)$	$\bar{\alpha}(n) = 1 + \bar{\mathbf{g}}^H(n)\mathbf{x}(n)$
A posteriori error	$\varepsilon(n) = \alpha(n)e(n)$	$\varepsilon(n) = \dfrac{e(n)}{\bar{\alpha}(n)}$
Coefficient updating	$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{g}(n)e^*(n)$	$\mathbf{w}(n) = \mathbf{w}(n-1) + \bar{\mathbf{g}}(n)\varepsilon^*(n)$

Algoritmos LS recursivos

El costo computacional de LS recursivo a priori y a posteriori es $O(M^3)$ por actualización, debido al cálculo de la ganancia de adaptación.

En LS a priori:

$$\mathbf{g}(n) = \hat{\mathbf{R}}^{-1}(n)\mathbf{x}(n)$$

En LS a posteriori:

$$\bar{\mathbf{g}}(n) = \lambda^{-1}\hat{\mathbf{R}}^{-1}(n-1)\mathbf{x}(n)$$

RLS convencional

El algoritmo **RLS convencional** reduce la complejidad computacional, actualizando la inversa de la matriz de correlación $\hat{\mathbf{R}}(n)$:

$$\mathbf{P}(n) = \hat{\mathbf{R}}^{-1}(n)$$

RLS

RLS convencional

Para encontrar una regla para actualizar $\mathbf{P}(n)$,
aplicamos el lema de inversión matricial:

$$(\lambda^{-1}\hat{\mathbf{R}} + \mathbf{x}\mathbf{x}^H)^{-1} = \lambda^{-1}\hat{\mathbf{R}}^{-1} - \frac{(\lambda^{-1}\hat{\mathbf{R}}^{-1}\mathbf{x})(\lambda^{-1}\hat{\mathbf{R}}^{-1}\mathbf{x})^H}{1 + \lambda^{-1}\mathbf{x}^H\hat{\mathbf{R}}^{-1}\mathbf{x}}$$

RLS convencional

RLS

Uniendo la igualdad anterior, la recursión de $\hat{\mathbf{R}}(n)$, la expresión de $\mathbf{g}(n)$ y la expresión del factor de conversión $\bar{\alpha}(n)$, obtenemos la ecuación de actualización de $\mathbf{P}(n)$:

$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\bar{\mathbf{g}}^{H}(n)$$

RLS convencional

RLS

Dadas $\mathbf{P}(n-1)$ y las nuevas observaciones $\{\mathbf{x}(n), y(n)\}$, podemos actualizar $\mathbf{P}(n)$ siguiendo estos pasos:

$$\bar{\mathbf{g}}(n) = \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)$$

$$\bar{\alpha}(n) = 1 + \bar{\mathbf{g}}^{H}(n)\mathbf{x}(n)$$

$$\mathbf{g}(n) = \frac{\bar{\mathbf{g}}(n)}{\bar{\alpha}(n)}$$

$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\bar{\mathbf{g}}^{H}(n)$$

RLS convencional

Este procedimiento define el algoritmo **RLS convencional**.

El algoritmo es válido tanto para filtrado lineal como para procesamiento de arreglos ya que no supone nada sobre los datos de entrada.

RLS

RLS convencional

RLS

Siguiendo un raciocinio similar, también podemos desarrollar una forma recursiva del SSE mínimo:

$$
\begin{aligned}
E_o(n) &= \lambda E_o(n-1) + \varepsilon(n)e^*(n) \\
&= \lambda E_o(n-1) + \bar{\alpha}(n)|\varepsilon(n)|^2 \\
&= \lambda E_o(n-1) + \frac{|e(n)|^2}{\alpha(n)}
\end{aligned}
$$

RLS convencional

El costo computacional de RLS convencional es $O(M^2)$ por actualización, debido al cálculo de $\tilde{\mathbf{g}}(n)$ y de $\mathbf{P}(n)$.

RLS

Inicialización de RLS convencional

Resta definir los valores iniciales $\mathbf{P}(-1)$ y $\mathbf{w}(-1)$ para inicializar el algoritmo RLS convencional.

Existen dos formas de hacer esto.

Inicialización de RLS convencional

La forma más obvia consiste en recolectar un bloque de datos $\{\mathbf{x}(n), y(n)\}$ para $-n_0 < n < -1$, con $n_0 > M$, y calcular $\mathbf{P}(-1)$ y $\mathbf{w}(-1)$ en forma exacta.

Inicialización de RLS convencional

En la práctica, sin embargo, solemos usar $\mathbf{P}(-1) = \delta^{-1}\mathbf{I}$, en donde δ es un valor pequeño del orden de $0.01\ \sigma^2_x$.

También solemos inicializar el vector de coeficientes en $\mathbf{w}(-1) = \mathbf{0}$.

Inicialización de RLS convencional

Es posible probar que este enfoque minimiza la función de costo:

$$E(n) = \sum_{j=0}^{n} \lambda^{n-j} |e(j)|^2 + \delta \lambda^n \|\mathbf{w}(n)\|^2$$

Esto se corresponde con una **regularización** de la solución LS, con parámetro δ.

Inicialización de RLS convencional

Para $n > M$, la matriz de las ecuaciones normales será $\delta\lambda^n\mathbf{I} + \hat{\mathbf{R}}(n)$, por lo que $\mathbf{w}(n)$ tendrá bias.

Sin embargo, cuando n es grande y $\lambda < 1$, el valor particular de δ no tiene importancia ya que RLS tiene memoria exponencialmente decreciente.

Resumen

Initialization

$\mathbf{w}(-1) = \mathbf{0} \qquad \mathbf{P}(-1) = \delta^{-1}\mathbf{I}$

δ = small positive constant

For each $n = 0, 1, 2, \ldots$ compute:

Adaptation gain computation

$$\bar{\mathbf{g}}(n) = \lambda^{-1}\mathbf{P}(n-1)\mathbf{x}(n)$$

$$\bar{\alpha}(n) = 1 + \bar{\mathbf{g}}^{H}(n)\mathbf{x}(n)$$

$$\mathbf{g}(n) = \frac{\bar{\mathbf{g}}(n)}{\bar{\alpha}(n)}$$

$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \mathbf{g}(n)\bar{\mathbf{g}}^{H}(n)$$

Filtering

$$e(n) = y(n) - \mathbf{w}^{H}(n-1)\mathbf{x}(n)$$

Coefficient updating

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{g}(n)e^{*}(n)$$

Estabilidad numérica de RLS convencional

Según la teoría de RLS convencional, en todo momento se han de cumplir las siguientes condiciones:

- La matriz $\mathbf{P}(n)$ es hermítica y positiva definida.
- El factor de conversión vale $0 < \alpha(n) \leq 1$, ya que la energía del error a posteriori es siempre menor a la energía del error a priori.
- La energía $E(n)$ es positiva.

Estabilidad numérica de RLS convencional

Sin embargo, debido a que usamos precisión finita, pueden surgir **inconsistencias numéricas** en la práctica.

Es posible probar que la principal causa de la inestabilidad numérica de RLS convencional es la actualización de $\mathbf{P}(n)$.

Estabilidad numérica de RLS convencional

En caso de surgir problemas de estabilidad numérica, debemos hacer cumplir la propiedad hermítica de $\mathbf{P}(n)$.

Para lograr esto, después de actualizar $\mathbf{P}(n)$, podemos:

- Calcular la parte triangular superior (o inferior) de $\mathbf{P}(n)$, y completar los valores faltantes con la propiedad $p_{ij}(n) = p_{ji}^{*}(n)$.
- O bien, reemplazar $\mathbf{P}(n)$ por $[\mathbf{P}(n) + \mathbf{P}^{H}(n)] / 2$.

Convergencia de RLS convencional

Para analizar la **convergencia** de RLS convencional, asumiremos que la señal deseada $\mathbf{y}(n)$ es generada por el modelo:

$$\mathbf{y} = \mathbf{X}\mathbf{h} + \mathbf{v}$$

Tomaremos los mismos supuestos que tomamos para el análisis estadístico de LS.

Convergencia de RLS convencional

Con $\lambda = 1$, $n > M$, la solución $\mathbf{w}(n)$ es idéntica a la solución de Block LS que vimos antes.

Por tanto converge, en la **media**, al vector de parámetros verdadero:

$$E\{\mathbf{w}(n)\} = \mathbf{h}$$

Convergencia de RLS convencional

Con $\lambda = 1$, $n > M$, es posible probar que la **matriz de correlación** de $\tilde{\mathbf{w}}(n) = \mathbf{w}_o$ - $\mathbf{w}(n)$ es:

$$\mathbf{K}(n) = E_o \frac{1}{n - M} \mathbf{R}^{-1}(n)$$

Por tanto, converge a $\mathbf{0}$ conforme aumenta n.

Convergencia de RLS convencional

Con $\lambda = 1$, $n > M$, el **desvío cuadrático medio** de $\mathbf{w}(n)$ es:

$$\mathscr{D}(n) = \mathrm{tr}\{\mathbf{K}(n)\} = \frac{E_o}{n - M} \sum_{i=1}^{M} \frac{1}{\lambda_i}$$

λ_i son los autovalores de $\mathbf{R}(n)$.

Por tanto, converge a 0 conforme aumenta n.

Notamos que el desvío cuadrático medio se magnifica por el autovalor de $\mathbf{R}(n)$ más pequeño.

Convergencia de RLS convencional

Con $\lambda = 1$, $n > M$, es posible probar que el **error de exceso** es:

$$E_{ex}(n) = \frac{M}{n - M} E_o$$

Por tanto, converge a 0 conforme aumenta n.

Convergencia de RLS convencional

Con $\lambda < 1$, es posible probar que $\tilde{\mathbf{w}}(n) = \mathbf{w}_o - \mathbf{w}(n)$ converge en la **media** a:

$$E\{\tilde{\mathbf{w}}(n)\} = \delta\lambda^n E\{\hat{\mathbf{R}}^{-1}(n)\}\tilde{\mathbf{w}}(0)$$

Conforme aumenta n, $\tilde{\mathbf{w}}(n)$ converge a $\mathbf{0}$ y $\mathbf{w}(n)$ converge al valor verdadero $\mathbf{h}$.

Convergencia de RLS convencional

Con $\lambda < 1$, es posible probar que la **matriz de correlación** de $\tilde{\mathbf{w}}(n) = \mathbf{w}_o - \mathbf{w}(n)$, en régimen estacionario, tiende a:

$$\mathbf{K}(\infty) \approx \frac{1-\lambda}{1+\lambda} E_o \mathbf{R}^{-1}$$

Por tanto **no** converge a $\mathbf{0}$.

Esto se debe a que RLS tiene memoria finita, y efectivamente no utiliza todos los datos.

Convergencia de RLS convencional

Con $\lambda < 1$, el **desvío cuadrático medio** de $\mathbf{w}(n)$, en régimen estacionario, es:

$$\mathscr{D}(\infty) = \mathrm{tr}\{\mathbf{K}(\infty)\} \approx \frac{1-\lambda}{1+\lambda} E_o \sum_{i=1}^{M} \frac{1}{\lambda_i}$$

Por tanto **no** converge a $\mathbf{0}$.

Notamos que el desvío cuadrático medio se magnifica por el autovalor de $\mathbf{R}$ más pequeño.

Convergencia de RLS convencional

Con $\lambda < 1$, es posible probar que el **error de exceso**, en régimen estacionario, es:

$$E_{ex}(\infty) \approx \frac{1-\lambda}{1+\lambda} M E_o$$

Por tanto, disminuye conforma aumenta λ.

Nuevamente observamos el efecto de la memoria finita de RLS.

Summary of RLS and LMS performance in a stationary SOE.

Property	Growing memory RLS algorithm ($\lambda = 1$)	Exponential memory RLS algorithm ($\lambda < 1$)	LMS algorithm
Convergence in the mean	For all $n > M$	Asymptotically for $n \to \infty$	Asymptotically for $n \to \infty$
Convergence in MS	Independent of the eigenvalue spread	Independent of the eigenvalue spread	Depends on the eigenvalue spread
Excess MSE	$E_{ex}(n) = \frac{ME_o}{n-M} \to 0$	$E_{ex}(\infty) \approx \frac{1-\lambda}{1+\lambda} ME_o$	$J_{ex}(\infty) \approx \mu J_o \text{tr}\{\mathbf{R}\}$

Comparación

Curva de aprendizaje

En RLS solemos definir la **curva de aprendizaje** $E(n)$ a partir del error a priori $e(n)$:

$$E(n) = E\{|e(n)|^2\}$$

Curva de aprendizaje

Cuando $\lambda = 1$, $n > M$, $E(n)$ es:

$$E(n) = E_o + E_{ex}(n) = E_o + \frac{M}{n - M} E_o$$

La convergencia se alcanza en aproximadamente $2M$ pasos.

Curva de aprendizaje

Cuando $\lambda < 1$, es posible probar que la forma de $E(n)$ depende de la SNR de los datos de entrada $\mathbf{x}(n)$ al ruido $v(n)$ del modelo de señal deseada:

- Con SNR alta (> 30 dB), la convergencia es **muy rápida**, requiriendo alrededor de $2M$ pasos.
- Con SNR media (~ 10 dB), la convergencia es **más lenta**.
- Con SNR baja (< -10 dB), la convergencia es **muy lenta**.

Curva de aprendizaje

La elección óptima de δ también depende de la SNR.

Moustakides (1997) sugiere seleccionarlo con:

$$\delta = \sigma_x^2(1-\lambda)^\alpha$$

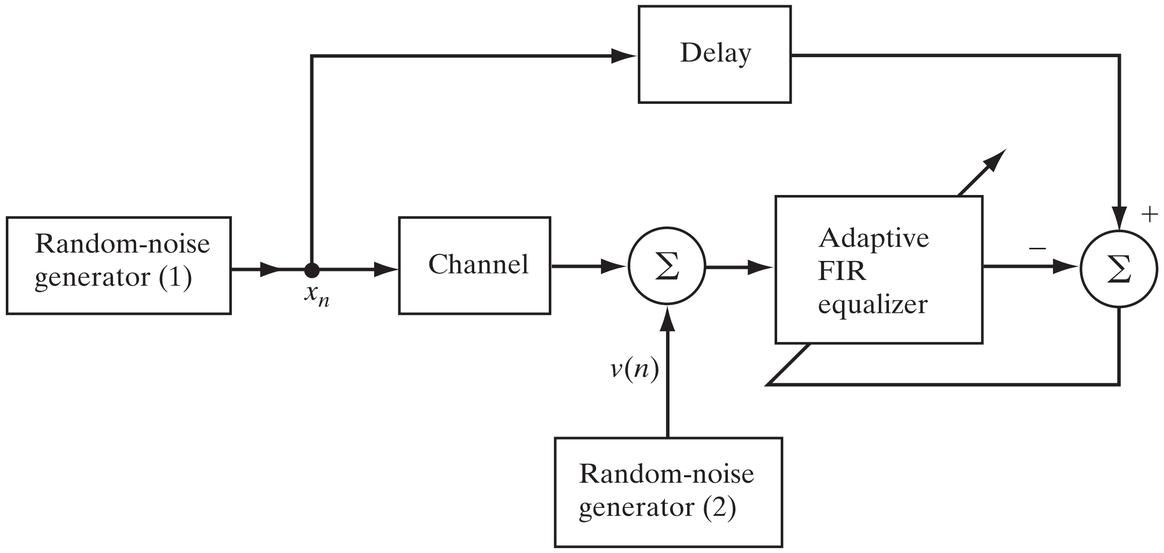
- Con SNR alta (> 30 dB), elegimos un valor $\alpha > 0$, típicamente $\alpha = 1$.
- Con SNR media (~ 10 dB), $-1 \leq \alpha < 0$.
- Con SNR baja (< 10dB), $\alpha < -1$.

Ejemplo

Para demostrar el comportamiento de RLS y compararlo con LMS, estudiemos el funcionamiento de un sistema de inversión de sistemas aplicado a un canal de comunicaciones.

Alimentamos el canal con una secuencia de Bernoulli x_n de valor ± 1.

Para simular el canal, añadimos ruido $v(n)$.

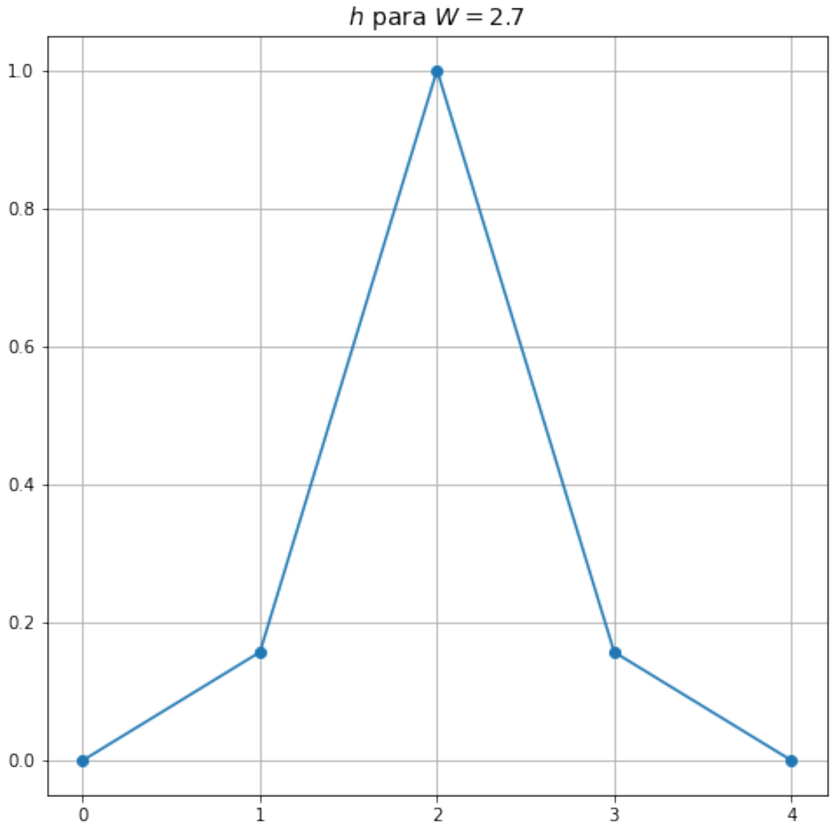


Ejemplo

Modelamos el canal con la respuesta al impulso:

$$h(n) = \begin{cases} 0.5\left\{1 + \cos\left[\frac{2\pi}{W}(n-2)\right]\right\}, & n = 1,2,3 \\ 0, & \text{sino} \end{cases}$$

Esta expresión permite controlar el spread de autovalores $\chi(\mathbf{R})$ a través del parámetro W.



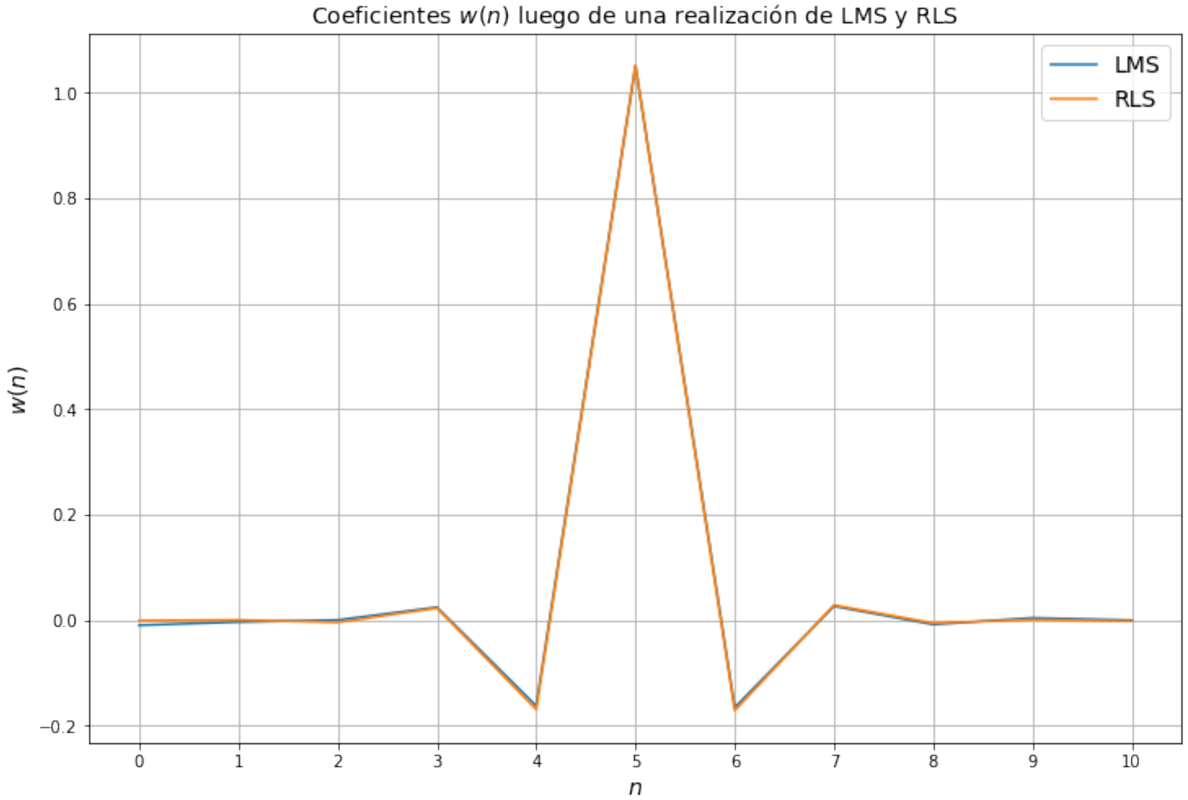
Ejemplo

Utilizamos un filtro de orden $M = 11$.

Dado que $\mathbf{h}(n)$ es simétrica, $\mathbf{w}(n)$ también lo será.

$\mathbf{h}(n)$ tiene un retardo medio de 2 taps, $\mathbf{w}(n)$ un retardo medio de 5 taps, por lo que utilizamos 7 taps para el bloque de retardo.

El ejemplo a la derecha compara el valor final de $\mathbf{w}(n)$ de una realización de LMS y de RLS.

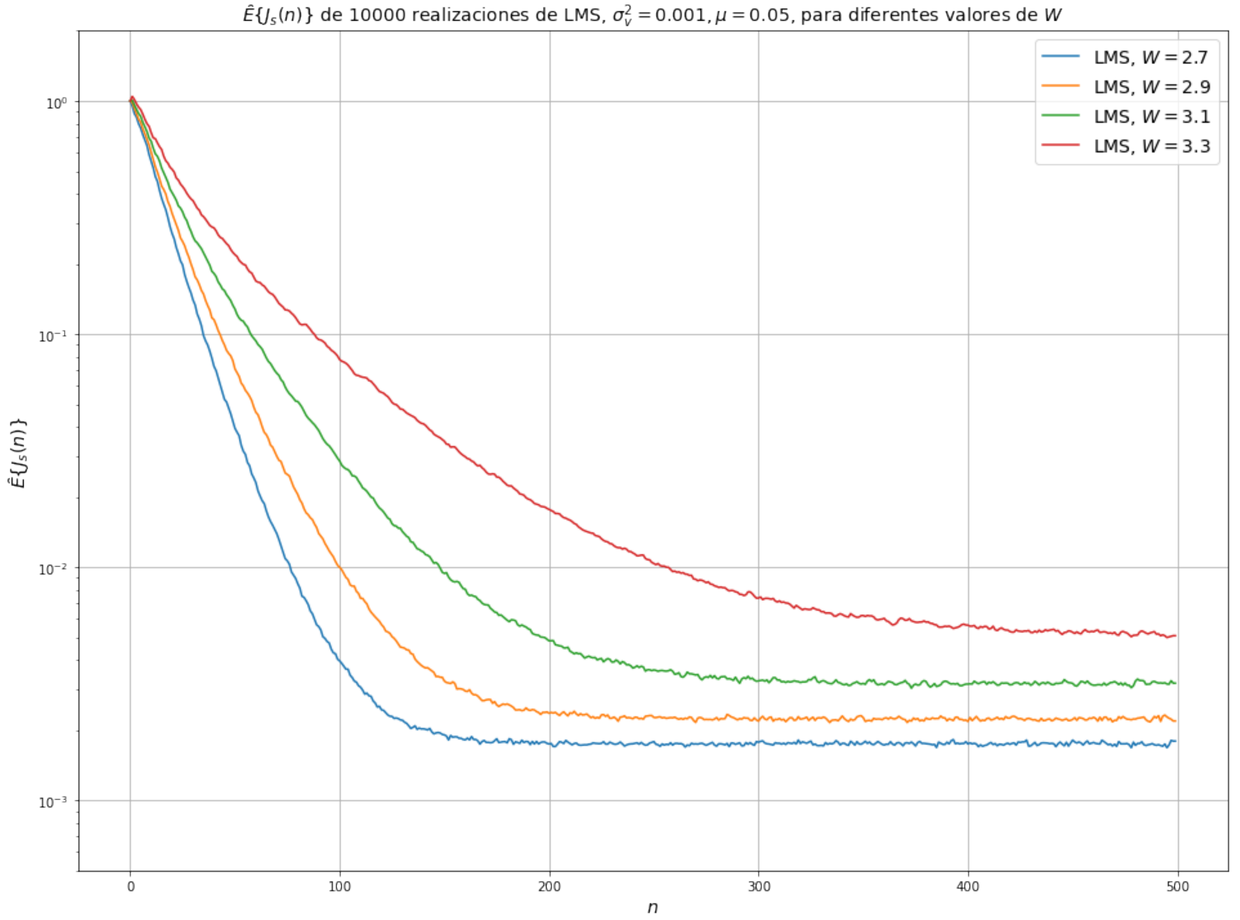


Ejemplo

Realicemos un primer experimento con $\sigma^2_v = 0.001$, que corresponde a un SNR de 30 dB.

Para el canal seleccionamos $W = \{2.7, 2.9, 3.1, 3.3\}$.

Comparemos 10000 simulaciones de Monte Carlo de LMS y de RLS.



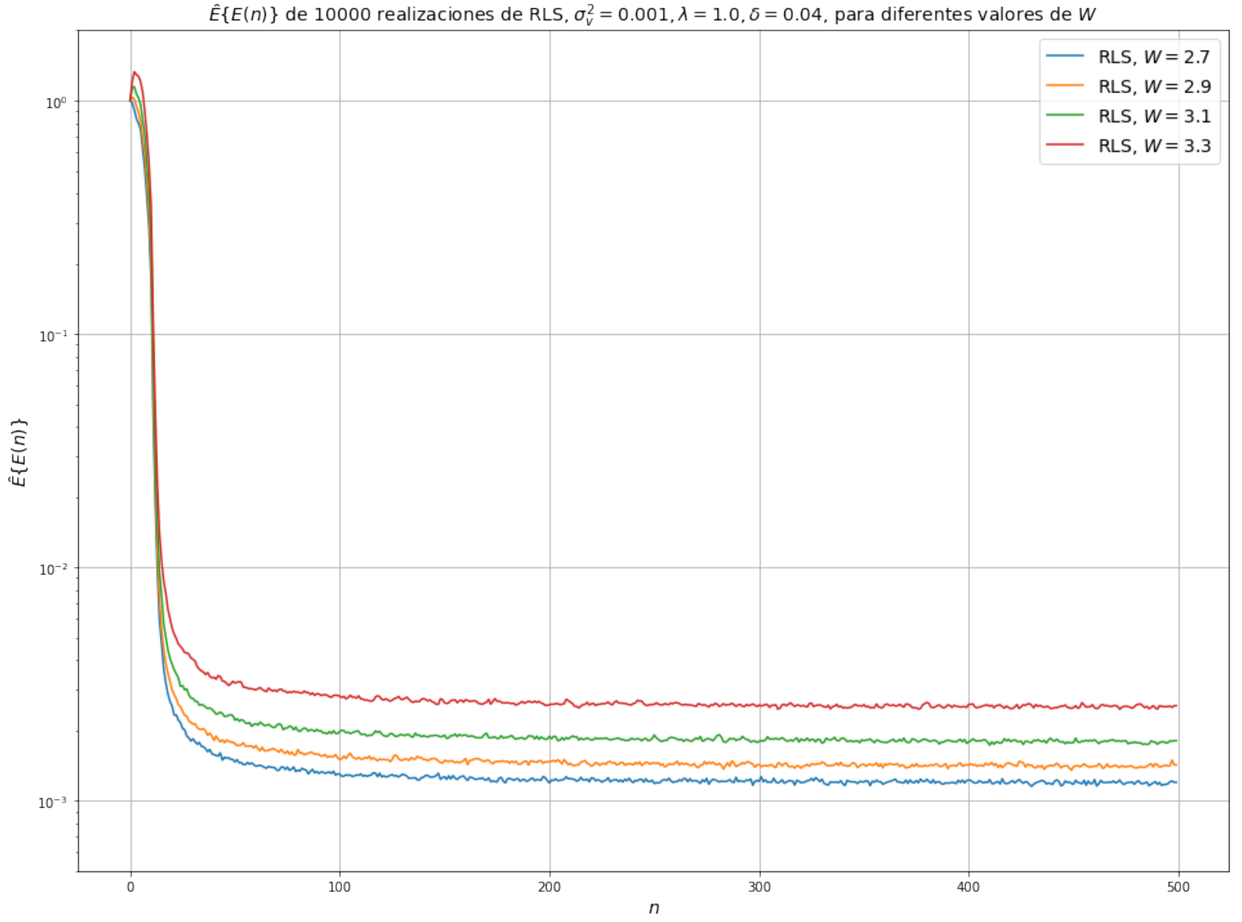
Ejemplo

En este ejemplo, RLS converge mucho más rápidamente que LMS.

De hecho, converge en aproximadamente 20 pasos (del orden de 2M).

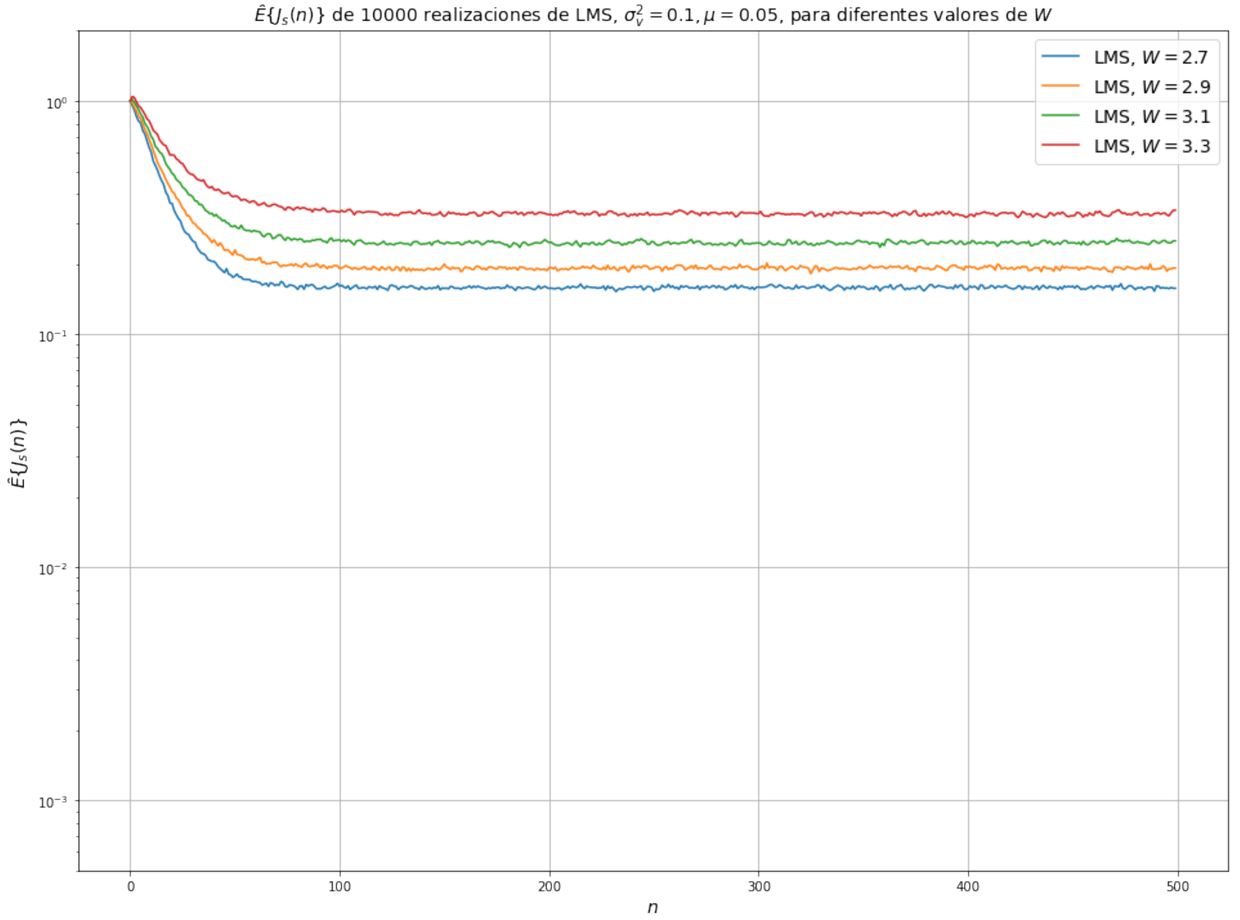
La tasa de convergencia es independiente del spread de autovalores.

El error estacionario de RLS es menor que el de LMS.



Ejemplo

En el segundo experimento seleccionamos $\sigma^2_v = 0.1$, que corresponde a un SNR de 10 dB.



Ejemplo

Ahora la convergencia de RLS y LMS es muy similar, requiriendo del orden de 40 pasos para converger.

